ENPM611 — Software Engineering | Final Project Presentation

Poetry Issues Analytics

A Multi-Feature Application of GitHub Issues for the Poetry Repository

Team

Chris Collins · Michael Hunt · Shiv Shah

15-minute presentation: 10 min talk + 5 min live demo

Overview

Approach and Design

Approach

- Waterfall Methodology (Separate Phases)
- Requirements
- Design
- Implementation
- Testing

Implementation

- config.json + env override for the data file path
- Two graph analyses within a feature
- One pure CLI analysis
- One interactive dashboard analysis

Design

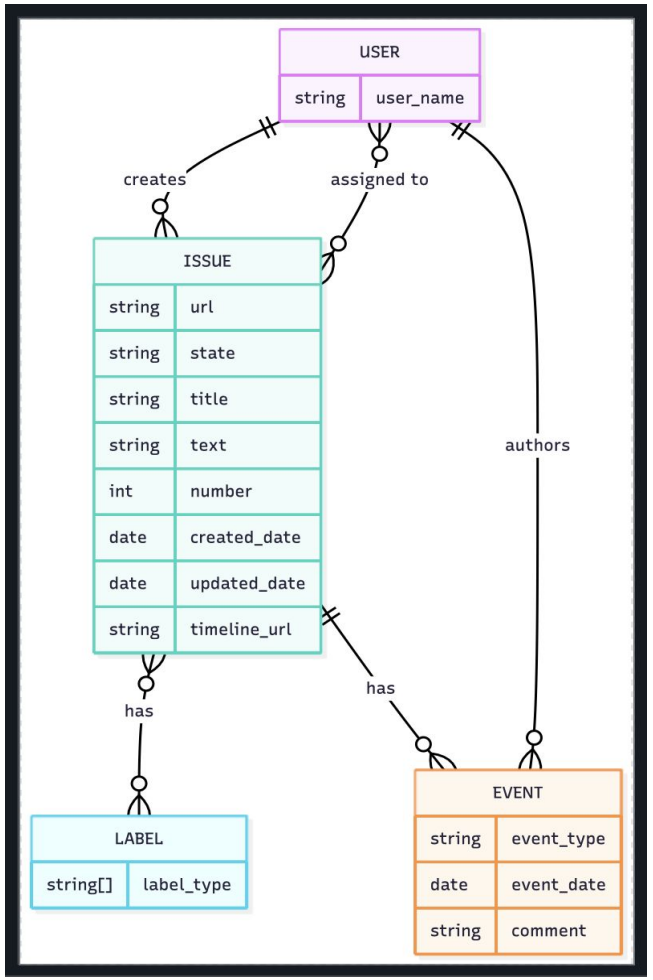
- DataLoader · config · model.Issue · model.Event
- IssuePull
- ExampleAnalysis · LabelsAnalysis · dashboard/
- run.py wires everything to a --feature flag

Testing

- Tests: Designed to achieve 90%+ code coverage across files

Initial Design (Milestone 1)

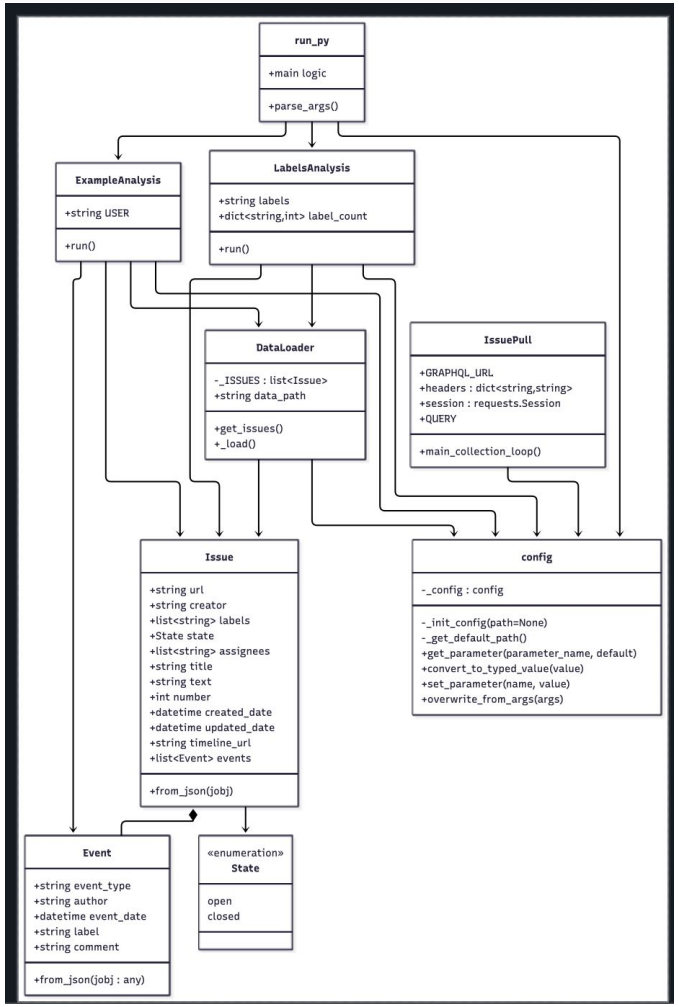
What we submitted: a class diagram + ERD for the CLI app



Plan at submission time

- Three CLI analyses, each printing tables + matplotlib charts
- Object-graph data model: Issue holds a list of Event
- config.json + env override for the data file path
- Tests: not yet scoped (Milestone 1 was design only)
- Diagrams committed to /diagrams as the source of truth

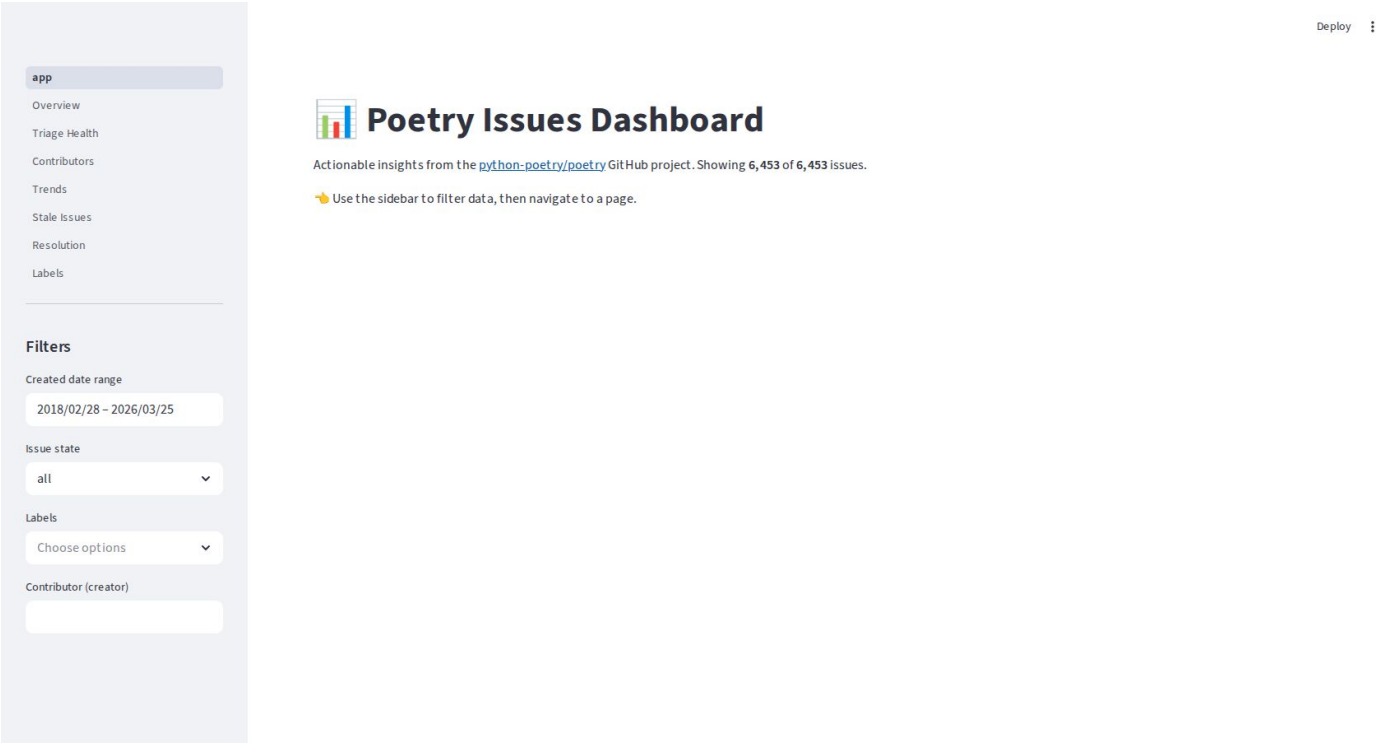
Note: there was no dashboard, no DataFrame layer, and no derived metrics in the original design.



Implementation

CLI analyses + a multi-page Streamlit dashboard over 6,453 GitHub issues

- 3 CLI features (run.py --feature 0/1/2): Example Analysis, Labels Analysis, State Analysis
- 2 graph analysis (run.py --feature 1): Label occurrences across issues and average time to close label
- 1 CLI output (run.py --feature 2): Closed/Open status of issues
- 1 dashboard feature (--feature 3): Launches Streamlit Interactive Dashboard
- 7-page dashboard: Overview, Triage, Contributors, Trends, Stale, Resolution, Labels
- Sidebar filters: date range, state, labels, contributor (apply globally)
- Computed metrics: time-to-first-response, days-to-close, staleness buckets



Dashboard landing page · live screenshot

What changed in the final design

Three categories of divergence from the milestone-1 submission

New Dashboard

- Streamlit dashboard module (dashboard/)
- 7 page modules under dashboard/pages/
- Sidebar filter module (filters.py)
- Metrics layer (metrics.py): 6 functions
- Cached data loader (st.cache_data)

New Class

- state_analysis.py added which contains the StateAnalysis class
- Labels analysis would now occur in it's own class with insights related solely to labels

Refined Requirements

- issue_pull.py not part of normal application usage
- Run once then utilize the data via config.json
- Normalized results, reduced running time, and circumvented API request limits

DEMO

Testing Suite

unittest and coverage modules for obtaining 90%+ code coverage

Coverage by layer

Dashboard (7 modules)

Loaded via runpy under a mocked streamlit; assert KPI labels underneath all the pages

Features (3 functions)

Heavy usage of function patching to only make the functions within the features run

Utilities (3 files)

Separately tested utilities as they were patched for features and dashboard testing. Optimized tests for code coverage.

Test Coverage Output

```
python -m coverage report --omit="test_*"
Name                                  Stmts   Miss  Cover
-------------------------------------------------------------
_page_test_utils.py                      50      3    94%
config.py                                68     24    65%
dashboard/__init__.py                     0      0   100%
dashboard/metrics.py                    135      8    94%
dashboard/pages/1_Overview.py            46      0   100%
dashboard/pages/2_Triage_Health.py       39      2    95%
dashboard/pages/3_Contributors.py        56      2    96%
dashboard/pages/4_Trends.py              29      3    90%
dashboard/pages/5_Stale_Issues.py        36      5    86%
dashboard/pages/6_Resolution.py          45      5    89%
dashboard/pages/7_Labels.py              29      3    90%
features/__init__.py                      0      0   100%
features/data_loader.py                  18      1    94%
features/labels_analysis.py              67      1    99%
features/state_analysis.py               29      1    97%
model.py                                 63      6    90%
-------------------------------------------------------------
TOTAL                                   710     64    91%
```

Bugs Discovered

Unexpected issues with code in main

Testing Utility Bug

The test utility had a bug in how it located the root directory, which caused path resolution problems when running tests.

→ Fix: Updated the utility to search for the project root directory during test execution.

Wrong divisor in LabelsAnalysis

Average time-to-close per label divides the per-label time sum by the TOTAL number of closed issues, not by the count of closed issues with that label.

→ Fix: Compute average close time per label by dividing each label's accumulated close time by that label's own closed-issue frequency.

Failed Configuration Load

The configuration loader is not reading the expected JSON config value during test execution (file/path loading path mismatch)

→ Fix: Ensure config.py loads from the project config.json in test context and that file-based values are merged before parameter reads.

Lessons Learned

Three honest answers — and the lesson behind each

1. Initial Design Scoping is Difficult

Milestone 1 asked for entity relationships and class structure, so we delivered exactly that. We never asked the next question — "what additional features might we want to implement" — until we started building. Sticking to the design diagram during implementation felt more restrictive than useful.

→ Lesson: Look beyond the milestone deadline to make a decision on what will be implemented and how the design will be done.

2. GitHub Issues Tracking is Super Useful

Before Milestone 2 we were coordinating all of our efforts via calls and text messages. It quickly became difficult to keep track of who was doing what and what needed to get done.

→ Lesson: GitHub Issues exists for a reason. This made tracking progress and milestone related tasks so much easier.

3. Testing May Result in Restructure

Our goal with Milestone 3 was to write tests such that 90%+ code coverage would be achieved. It turns out that the current code base wasn’t easily testable, requiring file structure and functions to be moved around to enable testing and achieve coverage.

→ Lesson: Consider entry points for testing when writing code.